

WePledge

Crowdfunding descentralizado com confiança programada

Blockchain

Smart Contract

DeFi

Ethereum · Solidity · Angular · Ethers.js

O Problema

CROWDFUNDING TRADICIONAL — POR QUE O MODELO ESTÁ QUEBRADO



Custódia centralizada

A plataforma detém os fundos. Ela pode bloquear saques, congelar campanhas ou encerrar operações — e o contribuinte não tem como reagir. Você confia na empresa, não no código.



Taxas de 5 a 10%

Uma fatia significativa de cada campanha vai obrigatoriamente para a plataforma. Não há negociação: é pagar ou ficar de fora. O beneficiário é o intermediário, não o projeto.



Regras que mudam sem aviso

Políticas de saque, critérios de elegibilidade e limites geográficos são definidos e alterados unilateralmente pela plataforma. Criadores e doadores não têm poder de contestação.



Opacidade total

Para onde vai o dinheiro? Quem contribuiu? Quando os fundos foram movimentados? Nenhuma dessas perguntas tem resposta verificável por um terceiro. O intermediário é o único auditor.

~8%

DO DINHEIRO DE CADA CAMPANHA VAI PARA A PLATAFORMA, NÃO PARA O PROJETO

**Confiança
forçada**

VOCÊ NÃO TEM ESCOLHA: OU CONFIA NA EMPRESA OU NÃO PARTICIPA

A Solução — WePledge

O INTERMEDIÁRIO É O CÓDIGO. AS REGRAS SÃO O CONTRATO.



100% on-chain

Campanhas, contribuições, liberações e reembolsos são transações na blockchain Ethereum. Imutáveis, verificáveis por qualquer pessoa, a qualquer hora.



Vesting automático

Os fundos são liberados ao criador em parcelas por tempo (tranches). O criador define o cronograma; o contrato o executa sem intermediário.



Reembolso garantido por código

Se a meta não for atingida ou o criador abandonar o projeto, contribuintes resgatam seus ETH diretamente do contrato — sem precisar pedir para ninguém.



Transparência total

Toda contribuição, todo saque e toda mudança de estado são eventos públicos na blockchain. Qualquer um pode auditar o histórico completo da campanha.

0%

TAXA DE
PLATAFORMA

100%

DOS FUNDOS
CONTROLADOS PELO
SMART CONTRACT

Como Funciona

FLUXO COMPLETO DE UMA CAMPANHA — DA CRIAÇÃO AO ENCERRAMENTO



Smart Contract — Máquina de Estados

TRANSIÇÕES EXPLÍCITAS, AUDITÁVEIS ON-CHAIN, SEM ESTADO AMBÍGUO



PRINCÍPIOS DO CONTRATO



Sem estado ambíguo

Sempre exatamente um estado ativo por campanha.



Pull payment

Contribuintes puxam o reembolso — contrato nunca empurra batch.



Auditável via eventos

Toda transição emite evento indexado na blockchain.

Mecanismos de Proteção

VESTING POR TRANCHES E JANELA DE ABANDONO — DIFERENCIAIS TÉCNICOS DO WEPLDGE

Vesting por Tranches Anti-fraude

O criador define até **5 parcelas** com percentuais e tempos. Os fundos são liberados automaticamente pelo contrato conforme o cronograma — **sem precisar de aprovação** de ninguém, mas também **sem poder adiantar** antes do prazo de cada tranche.



↑ `t=0 · finalizarCampanha()`

↑ `t=30d · sacarTranche()`

A última tranche usa o saldo exato restante — elimina dust de arredondamento preso no contrato.

Proteção Contra Abandono Anti-rug

Após o prazo de captação, o criador tem uma **janela exclusiva** para finalizar. Se não agir, abre-se a janela de abandono — e qualquer pessoa pode acionar o reembolso coletivo.



↑ Início

↑ Prazo de captação

↑ Abandono liberado

Produção: 7 dias (finalização) + 30 dias (abandono). Ambos imutáveis após o deploy — ninguém pode mudar as regras depois.

Segurança do Smart Contract

PADRÕES E GARANTIAS IMPLEMENTADOS NO WEPLIDGE



ReentrancyGuard (OpenZeppelin) + padrão CEI

Funções com transferência ETH (sacarTranche, reembolsar) usam mutex global + Checks-Effects-Interactions: estado atualizado antes de qualquer call externa. Re-entrada encontra estado já modificado e reverte.



Pull payment — sem batch refund

Cada contribuinte chama reembolsar() individualmente. Elimina o vetor clássico de batch-refund (loop pode exceder gas limit do bloco ou falhar se um endereço reverter, bloqueando todos os outros).



Parâmetros imutáveis pós-deploy

JANELA_FINALIZACAO, JANELA_ABANDONO, MAX_TRANCHES e MAX_PRAZO_CAPTACAO são declarados immutable — lidos do bytecode, não do storage. Impossível alterar as regras após o deploy.



Sem receive() / fallback() útil — ETH só entra via contribuir()

Envio direto ao contrato reverte com mensagem explícita. Elimina ETH não-rastreado preso no contrato sem mecanismo de resgate.



Sentinela de campanha inexistente

ID 0 reservado; campanhas reais começam em 1. Verificação de existência via criador != address(0) — sem campo extra de storage.



call(value) em vez de transfer/send

transfer e send limitam o gas a 2300 (stipend EIP-1884), quebrando contratos com fallback não-trivial. call repassa gas suficiente; retorno booleano verificado explicitamente.



Aritmética segura (Solidity 0.8.24)

Overflow e underflow causam revert automático desde Solidity 0.8. Sem SafeMath explícito — proteção nativa do compilador. Checked arithmetic por padrão.



Suite de testes com Hardhat + Ethers.js

Cobertura de todos os estados, transições e casos-limite — prazo, meta, abandono, reembolso, saque duplo, reentrância. Testes rodam em fork de rede local (Hardhat Network).

Diferenciais do WePledge

O QUE SEPARA ESTE PROJETO DE UMA PLATAFORMA DE CROWDFUNDING COMUM

1

Zero intermediários

Nenhuma empresa controla os fundos. O contrato é o intermediário — e ele segue apenas as regras escritas em código, sem exceções, sem subjetividade.

2

Vesting forçado com cronograma

Criador não recebe tudo de uma vez. Cada tranche é um checkpoint: ele precisa agir para sacar. Accountability programada — sem depender de promessas.

3

Proteção anti-rug automática

Se o criador desaparece com a meta atingida, a comunidade pode acionar o reembolso via `marcarAbandono()`. Nenhuma plataforma centralizada precisa agir.

4

Reembolso garantido por código

Em campanha fracassada ou abandonada, qualquer contribuinte chama `reembolsar()` e recupera exatamente o que contribuiu. Sem formulário, sem suporte, sem espera.

5

Histórico imutável e público

Todos os eventos — criação, contribuição, meta atingida, tranches, reembolsos — são registros permanentes na blockchain, auditáveis por qualquer observador externo.

6

Interface acessível sem carteira

Qualquer pessoa pode navegar, visualizar campanhas e acompanhar o progresso sem precisar conectar MetaMask. Provedor de leitura separado do provedor de escrita.

WePledge

Confiança programada.

"As regras são o código.

A lei é o contrato.

*O intermediário é **desnecessário**."*

Solidity 0.8.24

Hardhat

Ethers.js v6

Angular 19

Sepolia Testnet

SMART CONTRACT



Solidity

Máquina de estados, vesting, abandono, reembolso

0.8.24

DESENVOLVIMENTO



Hardhat

Compilação, deploy, testes locais, fork de rede

Local + Sepolia

BIBLIOTECA WEB3



Ethers.js

Interação com contrato, eventos, providers

v6

FRONTEND



Angular

Signals, lazy-loading, standalone components

v19